

Epistemic Half-Life

Truth Decay Semantics for Stored Data

P-RICK RESEARCH PROGRAM

an independent working-paper series on missing software

working paper P-018 v1.0 • series V

October 7, 2026 • cc by 4.0

Abstract — Every database ever deployed rests on a single unexamined premise: that a stored fact remains true. It does not. Addresses silently migrate, phone numbers churn, prices drift by the hour, allergy records are superseded, credentials expire, “current” employer fields fossilize, and the record of what a device’s firmware could do is falsified by next year’s patch. Food answered this problem with a sell-by date; medicine with lot traceability; journalism with corrections — and software, the industry that stores more facts about the world than all of them combined, answered it with nothing. The database tuple has no shelf life, the query planner has no notion of staleness, and the join operator is structurally incapable of noticing that it is joining a 2020 address to a 2025 delivery order. This paper specifies the missing semantics: facts carry decay functions (with half-life as the first-class parameter), confidence decays from capture through verification to deprecation, queries operate over (value, confidence) pairs with explicit composition rules — the stale join inherits the worst of its inputs — and a verification scheduler budgets the re-probing of aging facts the way schedulers budget any scarce resource. The half-lives themselves are estimable from an organization’s own update history: the empirical-Bayes insight that every enterprise already logs, in its change streams, the decay rates it refuses to model. The gap survives because the relational model made tuples eternal, because staleness is a content property that fell between schema and application teams, and because no query semantics existed to make decay worth modeling. Every component exists in an adjacent world — bitemporal belief tracking, cache freshness, pipeline freshness tests, decay-rate lore in finance and CRM — and no system composes them into general, query-level, propagatable truth decay. The claim is stated to be falsified by a single system that does. **Keywords:** staleness, truth decay, half-life, freshness, bitemporal, data quality, query semantics, verification scheduling

Keywords: staleness, truth decay, half-life, freshness, bitemporal, data quality, query semantics, verification scheduling

p-rick working paper P-018 · series V (promises) · draft 1.0

Every database ever deployed rests on a single unexamined premise: that a stored fact remains true. It does not. Addresses silently migrate, phone numbers churn, prices drift by the hour, allergy records are superseded, credentials expire, “current” employer fields fossilize, and the record of what a device’s firmware could do is falsified by next year’s patch. Food answered this problem with a sell-by date; medicine with lot traceability; journalism with corrections — and software, the industry that stores more facts about the world than all of them combined, answered it with nothing. The database tuple has no shelf life, the query planner has no notion of staleness, and the join operator is structurally incapable of noticing that it is joining a 2020 address to a 2025 delivery order. This paper specifies the missing semantics: facts carry decay functions (with half-life as the first-class parameter), confidence decays from capture through verification to deprecation, queries operate over (value, confidence) pairs with explicit composition rules — the stale join inherits the worst of its inputs — and a verification scheduler budgets the re-probing of aging facts the way schedulers budget any scarce resource. The half-lives themselves are estimable from an organization’s own update history: the empirical-Bayes insight that every enterprise already logs, in its change streams, the decay rates it refuses to model. The gap survives because the relational model made tuples eternal, because staleness is a content property that fell between schema and application teams, and because no

query semantics existed to make decay worth modeling. Every component exists in an adjacent world — bitemporal belief tracking, cache freshness, pipeline freshness tests, decay-rate lore in finance and CRM — and no system composes them into general, query-level, propagatable truth decay. The claim is stated to be falsified by a single system that does.

1 Introduction

A package is undeliverable, three thousand kilometers from its destination, because a checkout form joined a customer’s address from 2019 to an order from 2026 — and no layer of the stack, from the form through the ORM to the database, possessed a representation of the fact that one of those two facts had not been re-verified in seven years. A hospital’s emergency-contact field, captured at admission, silently rots across a decade of moves and deaths — and presents itself with exactly the same visual and logical authority as the patient’s blood type. A compliance dashboard marks a vendor “certified” in 2023 and keeps marking it certified through the certification’s expiry, the vendor’s acquisition, and the certification body’s own dissolution, because certification was modeled as a state rather than as a perishable claim. In each case the data is *available, consistent, and wrong* — the failure mode no integrity constraint detects.

These are not exotic failures. They are the routine, quiet, compounding errors of every operational database, and the industry’s response has been to treat them as someone else’s layer. The ETL team calls it data quality; the CRM industry calls it contact decay and sells list-cleaning services against it (the B2B marketing literature quotes contact-data decay of a few percent *per month* — a figure this paper treats as domain lore marked *verification queued*, but one that practitioners repeat with the weary certainty of people who bill for it); the security team calls it credential hygiene; and the database — the one artifact common to all of them — models none of it. The tuple that was true when written is served with identical confidence on day zero and day four thousand, because Codd’s relational model, for all its rigor, has exactly one temporal dimension baked into its soul: none.

The deeper observation is that decay is *measurable in advance*. Facts do not rot at random; they rot by class. A stock quote has a half-life measured in seconds; a retail price, in weeks; an address, in years; a blood type, effectively never. An organization that has run for five years possesses, in its own update logs, the empirical decay curve of every fact class it maintains — the actuarial table of its own data — and it possesses it by accident, discarding it as operational exhaust. The half-life of a fact class is not a mystery; it is a statistic sitting in a change-data-capture stream that nobody has ever aggregated, because no query semantics existed that would make the statistic worth computing. This paper supplies the semantics, and the statistics become load-bearing.

Section 2 formalizes decay and distinguishes it carefully from the adjacent temporal machinery (bitemporality models *belief*, not *truth*). Section 3 surveys the fragments — bitemporal databases, TTL caches, HTTP freshness, pipeline freshness tests, finance’s stale-quote discipline — and verifies the gap. Section 4 explains the survival of the gap with reference to the relational model’s history. Section 5 specifies the system: decay declarations, decay-function families, query semantics with composition rules, join-decay algebra, the verification scheduler and its budget, decay events, the decay ledger, and compaction. Sections 6–10 deliver the evaluation design, boundary claims, objections, limitations, and conclusion. This is series V’s second paper: *promises software implies and never has to honor* — here, the promise that a stored fact is a fact.

2 A theory of truth decay

2.1 Decay functions and half-life

For a fact f — an atomic (entity, attribute, value, t_{capture}) tuple — define a **decay function** $d_f : \text{time} \rightarrow [0, 1]$, the confidence that f 's value still holds at query time. The first-class parameterization is the **half-life** h : exponential decay, $d_f(\Delta t) = 2^{-(\Delta t/h)}$, for the broad class of facts whose hazard of invalidation is roughly constant per unit time (addresses, prices, contact details). Two other families cover the tails: **step decay** ($d = 1$ until a known expiry, then 0 — certifications, licenses, medication dosages with clinical review dates), and **hazard-estimated decay** (d derived from the observed update distribution of the fact's class — Section 5.2), which generalizes both and is the estimator that the change-stream statistics feed.

The half-life is the right first-class constant for three reasons. It is human-interpretable (“addresses: ~ 4 years” is a sentence an administrator can argue with); it composes (Section 2.3); and it is estimable from update histories without domain committees, which matters because the political economy of asking every department to declare decay rates is the political economy of asking every department to volunteer work — the estimate from their own logs asks them nothing.

2.2 Decay is not belief time: the bitemporal boundary

The adjacent machinery must be kept honest, because the nearest neighbor is genuinely near. Bitemporal databases (Snodgrass's tradition) record two times per fact: *valid time* (when the fact was true in the world) and *transaction time* (when the database believed it). They answer “what did we believe on date X about period Y” with archival rigor. What they do not and cannot express is a *probability that an unrefreshed belief is still true* — bitemporality is a ledger of belief changes, complete precisely because it records the changes that *arrived*. Staleness is about the changes that have *not yet arrived*: the address that has moved and not told us. A bitemporal database serves the 2019 address on January 1, 2026 with full transaction-time legitimacy — the belief is correctly recorded, its provenance immaculate, and its truth decaying invisibly. Decay semantics are thus not an extension of bitemporality but an orthogonal dimension: belief time is *retrospective*, truth decay is *predictive*. The composition of both — a decay-aware bitemporal store — is part of the specification (Section 5.6) precisely because the two dimensions answer different questions and the system needs both.

The same boundary applies to freshness machinery in delivery systems. HTTP cache freshness (RFC 7234) and Redis TTLs govern *whether a cached copy may be served without revalidation* — a delivery-layer contract between cache and origin. This paper's object is the *epistemics of the origin's own facts*: the question is not “may I serve this copy” but “is this fact still true, and what is our confidence, and who should re-verify it, when, at what budget.” The distinction sounds academic until one notices that a modern data platform's “freshness” tests (dbt source freshness, Great Expectations) are pipeline-latency monitors — they measure *when data last arrived*, a proxy that collapses entirely for slowly-changing dimensions, where the last arrival was years ago because most rows genuinely have not changed, and the monitor's green light means nothing about the rows whose world silently moved.

2.3 Composition: the algebra of decayed facts

Decay propagates through derived values, and the propagation rule is the paper's technical core:

- **Selection/aggregation over source facts:** confidence of the derived fact = the composition of input confidences — product for conjunction-style derivation, min for weakest-link aggregation (a mailing-list segment is as stale as its stalest address).
- **The stale join problem:** joining fact sets A and B yields pairs whose joint confidence = $d(A) \cdot d(B)$ by independence, but *real joins are rarely independent* — the 2019 address and the 2026 order share an entity whose maintenance cadence itself decays. The specification's join-decay algebra therefore

carries a per-entity **maintenance half-life** (how often the entity’s facts are touched at all), and the join planner composes entity-level and attribute-level decay explicitly rather than pretending independence.

- **Divergence:** when two sources assert different values for the same (entity, attribute), the system does not resolve the conflict silently; it records a **divergence event** with both confidences decayed to the moment of comparison. Conflict detection is decay’s most valuable side effect — most real-world data conflicts are staleness conflicts wearing a contradiction’s costume.

The algebra’s payoff: staleness stops being a monitoring dashboard’s opinion and becomes a *computable property of every query result*, which is the single move that makes every downstream behavior (display, blocking, verification scheduling) implementable.

2.4 The verification economy

Decay without verification is fatalism. The full model has three verbs per fact: **capture** (d resets to 1), **verify** (a probe that either resets d to 1 — fact confirmed — or supersedes the fact with a new capture), and **deprecate** (retire the fact from serving; archive it). Verification is not free: a probe costs an API call, a letter, a phone call, a nurse’s minute, a field technician’s afternoon. The interesting question — *which facts should be re-verified, in what order, under a budget* — is a scheduling problem over decay curves, with expected-information-gain per unit cost as the ranking function: the scheduler probes facts whose expected confidence lift $\times$ downstream dependency weight is highest. This is deliberately the attention-scheduler’s economics (P-006) relocated from human interruption to data verification: attention was the scarce resource there; probes are the scarce resource here, and the optimal-policy family (index policies, aging-item scheduling) transfers.

3 The landscape: fragments everywhere, semantics nowhere

Bitemporal databases and SQL:2011 temporal features. The mature nearest neighbor: belief and validity periods, system-versioned tables in mainstream engines. Retrospective, complete, and silent on predictive truth decay (Section 2.2). Verdict: solves the history of belief; does not touch the future of it.

TTL caches (Redis et al.) and HTTP freshness (RFC 7234). Delivery-layer freshness with clean revalidation semantics — the protocol machinery this paper’s verification verbs borrow, at the wrong layer and with no fact semantics. A cache that expires knows nothing about whether the origin’s fact was true; a Redis TTL on a customer record is a janitor, not an epistemologist. Verdict: right verbs, wrong object.

Pipeline freshness tests (dbt source freshness, Great Expectations, Monte Carlo/observability). The modern data stack’s honest admission that staleness matters — operationalized as *arrival latency* monitoring. Measures the pipe, not the water; blind to slowly-changing-dimension rot (Section 2.2). Verdict: a proxy metric with a dashboard, not query semantics.

Finance: stale quotes and market data. The deepest domain discipline: quotes carry timestamps, staleness bounds are hard-coded into matching engines, and “stale data” is a named regulatory concept in market infrastructure. Domain-bound, symbol-keyed, no general fact model; the discipline proves decay is *engineerable* under profit motive. Verdict: the existence proof, in one building.

CRM contact decay industry. An entire market (list hygiene, enrichment, “data enrichment decay” whitepapers) monetizing address/phone/email rot in B2B data. Rates quoted as a few percent per month (verification queued — the figures circulate through vendors selling the fix). The market exists *because* the databases have no decay semantics: decay is externalized to a cleaning service the way flood damage is externalized to insurance. Verdict: the gap, industrialized.

Master data management and entity resolution. Golden-record tooling with survivorship rules; strong on *conflict resolution at merge time*, weak on *time-passed-since-verification* as a survivorship criterion. Verdict: consolidation without half-lives.

Table 1: Table 1: the landscape, graded.

System	Time model	Decay function	Query semantics	Composition	Verification scheduler	General facts
Bitemporal / SQL:2011	belief + validity	none	temporal predicates	none	none	yes
TTL caches / HTTP	delivery age	step (expiry)	cache gating	none	revalidate-on-miss	delivery objects
Pipeline freshness	arrival latency	none	alerts	none	none	pipelines
Market data feeds	quote time	domain bounds	engine-enforced	domain rules	feed ops	symbols
CRM hygiene	none	none	none	none	bulk campaigns	contacts
MDM / entity resolution	merge time	none	survivorship	none	none	entities
GDPR accuracy	legal duty	none	none	none	none	personal data
This spec	belief + decay	half-life families	confidence-aware	join algebra	budgeted	all facts

Scientific data management. Curation metadata, dataset versions, README provenance — decay handled by retraction and versioning at the *document* level. The unit of rot in operational systems is the attribute, not the paper. Verdict: adjacent in spirit, different granularity.

GDPR Article 5(1)(d): the accuracy principle. Personal data must be “accurate and, where necessary, kept up to date.” The legal system named the duty in 2016 — and, characteristically, supplied no mechanism, no semantics, no measurement. Verdict: the promise, without the postal service (the recurring series-V shape: P-017 found the same silhouette under consent).

Components PARTIAL-to-STRONG across the row; composition STRONG-vacant. The falsifiable claim: no deployed system offers fact-level, query-propagated, scheduled-verification truth decay as a general semantic. One counterexample refutes it, and the search protocol for that counterexample is recorded in the program ledger.

4 Why the gap survived

The relational model’s eternity clause. Codd’s model fixed the time dimension at *snapshot*, and forty years of tooling, normalization theory, index structures, and personnel were trained on timeless tuples. Temporal extensions fought their way in over decades (and won a standard in 2011) by modeling *history* — the retrospective dimension, where the database is the authority. The predictive dimension — where the *world* is the authority and the database is a decaying sensor — never had a standardization beachhead, because it requires the database to express *uncertainty about the world*, a category the model’s set-theoretic foundations were never asked to carry. The result is an ontological blind spot inherited by every engine built on the lineage.

Staleness fell between the chairs. Schema teams own structure; application teams own behavior; and decay is a *content* property — a fact about the world’s churn rate per attribute class — that neither team’s charter mentions. The DBA cannot declare an address’s half-life (not a schema constraint); the application cannot enforce it per-join without re-implementing a query engine; and so both sides optimistically assume the other is handling it. Institutional gaps of exactly this shape — cross-cutting properties with no owning layer — are where this program keeps finding its territory (the same shape as defaults governance in P-016 and complexity accounting in P-012).

Storage got cheap, so deletion lost its urgency. The historical pressure that forced data lifecycle decisions was scarcity; abundance removed it. Decay is a *scarcity* discipline — half-lives matter when you must choose what to keep fresh — and in an era of effectively free storage, “keep everything forever” was always locally rational and globally a slow leak of falsehood into every downstream join.

Freshness proxies arrived first and satisfied the market. Pipeline-latency dashboards gave data teams a green light to show leadership, and the proxy — *data arrived recently* — felt like truth maintenance while measuring logistics. A satisfied market does not demand semantics; the dashboards are this gap’s most effective insurance policy.

Nobody estimated the half-lives because nothing needed them. The empirical Bayes estimator of Section 5.2 requires only update logs that every operational store already keeps — but there was no consumer for its output, so no one ever ran it. Semantics precede statistics here, not the reverse; the gap is an engineering order-of-operations accident, frozen by forty years of it.

5 The specification

5.1 Decay declarations

Schema-level, per (entity, attribute):

```
FRESH(entity.address, family = exponential, half_life = 4y, verify = mail |
api | human) FRESH(entity.certification, family = step, expires_at = attribute.certificate_
expiry) FRESH(entity.blood_type, family = hazard, estimator = update_history,
verify = clinical)
```

Declarations are versioned metadata (the defaults-ledger discipline: every decay declaration is itself a governed default with provenance, because a half-life is a policy someone set — P-016’s machinery composes directly). Defaults per type are seedable from the estimator; human declarations override; both are recorded in the decay ledger with who/when/why.

5.2 The estimator: half-lives from your own change stream

For each (entity-class, attribute), aggregate the interval distribution between successive updates across entities with at least two observed updates; fit the implied hazard rate λ ; the class half-life is $\ln 2 / \lambda$. The estimator is deliberately naive — empirical, not mechanistic — and deliberately *local to the organization*: a logistics company’s address half-life differs from a pediatric clinic’s, and the spec’s position is that the organization’s own history is the best prior available, superior to any industry table (the CRM vendors’ tables are the industry table, and they are the *competition’s* average, not your fleet’s). Entities with a single update (never refreshed) are censored observations — the estimator handles them via survival analysis rather than dropping them, because the never-refreshed population is precisely where decay hides.

5.3 Storage and query semantics

Each fact row carries (value, t_capture, t_last_verify, confidence) where confidence is *derived at read time* from the decay family and the elapsed interval — not stored and updated on a clock (a cron-driven confidence column is a correctness bug wearing a workflow’s clothes). The query surface:

- `SELECT ... WITH CONFIDENCE` returns (value, confidence) pairs.
- `WHERE CONFIDENCE(x) > 0.7` filters on freshness — the checkout form’s minimum viable hygiene: do not join an address below the delivery threshold.
- `JOIN ... ON MIN_CONFIDENCE` applies the join algebra of Section 2.3, with the maintenance half-life composition.
- Aggregates degrade honestly: `AVG(price)` over decayed inputs reports its confidence bound — composition with the uncertain-document’s arithmetic (P-015): measurement uncertainty and staleness uncertainty multiply into one reported bound, and the display contract of P-015 renders it. The two papers are one system at the surface: P-015 made numbers honest about *how they were measured*; this one makes them honest about *when*.

5.4 The verification scheduler

A budgeted scheduler over the decay ledger: inputs are per-fact decay curves, per-verb verification costs (api: ~0; mail: ~0.50; call: ~5; human: ~60 — currency-agnostic units), and downstream dependency weights (facts feeding payment routing or clinical decisions outrank facts feeding analytics). The policy class is an index policy: rank by expected confidence-lift $\times$ dependency weight / cost, probe within budget, capture the reset or the supersession. Escalation is automatic: facts below serving threshold that exhaust their cheap verification verbs escalate to expensive ones *only when* dependency weight justifies it — the attention-scheduler’s admission control, transposed (P-006). The scheduler’s honesty rule: it never silently fabricates verification (marking a fact verified because “it looked fine”) — the decay ledger records the verb or it records nothing, and anything else is the doctrine of fake freshness, which is worse than decay because it is decay with a forged timestamp.

5.5 Decay events

The serving threshold crossing is an *event*: `fact_stale(entity, attribute, confidence, consumers)` — published to consumers on the event bus (the P-001 fabric is the natural substrate; the paper’s contribution is the event *type* and its semantics). Consumers subscribe the way they subscribe to outages: a payment router freezes a beneficiary record pending verification; a mail-merge drops below-threshold addresses; a compliance dashboard turns amber. Staleness becomes an outage class — visible, routed, owned — rather than a private disappointment discovered by the undeliverable package.

5.6 The decay ledger and compaction

The ledger records per-fact capture/verify/supersede history with actor and verb — bitemporality’s transaction-time rigor (Section 2.2) retained as the audit spine, with the decay dimension as the predictive overlay. Compaction is the lifecycle’s quiet half: facts below archival confidence migrate to a cold tier where they are *unqueryable by default* (selectable only with explicit INCLUDING STALE), which is the storage-honesty discipline the era of free storage never developed: old data does not disappear, it *demotes*, and demotion is a semantic state, not a deletion. Privacy composes here: retention limits (GDPR storage-limitation) become an *upper bound on half-life* — decay is privacy legislation’s natural ally, and the same fabric that schedules verification also schedules minimization, one clock serving both duties.

5.7 Display and API contract

External surfaces (UI, partner APIs) receive confidence-annotated values under the calibrated display contract inherited from P-015: tiers (fresh / aging / stale / archived), rendered consistently, never fabricated precision. API consumers declare minimum confidence the way they declare timeouts; the contract is enforceable at the gateway. The end state is a lingua franca: “this address, confidence 0.31, last verified 2023-04” is a sentence both a courier’s routing engine and a nurse’s admission form can act on — and today, in every stack on earth, it is a sentence no system can say.

6 Evaluation design

1. **Staleness audit of real corpora.** Instrument three personal-scale and one enterprise-scale dataset (with owner consent): sample fact classes, verify ground truth, measure realized invalidation rates, and compare against estimator predictions. Success: estimator calibration within noise for slowly-changing classes.
2. **The stale-join experiment.** Reproduce the checkout failure synthetically: join orders to addresses across a 6-year window with and without decay semantics; measure misdelivery-rate proxies (join to superseded addresses) and the blocking rate of the confidence filter. Success: semantic blocking with false-block rate below the misdelivery rate by an order of magnitude.
3. **Scheduler economics.** Simulate a 10^6 -fact ledger with heterogeneous classes and a verification budget; compare the index policy against round-robin and age-only baselines on downstream-weighted confidence. Success: the information-gain policy dominates at all budgets.
4. **Display-contract legibility.** User study on confidence-tier rendering (inherited from P-015’s instrument): can untrained users correctly defer on stale values? Success: decisions track confidence tiers.

7 What this is not

Not bitemporality. The belief ledger is retained, not replaced; decay is the orthogonal predictive dimension (Section 2.2). **Not cache freshness.** Revalidation verbs are borrowed; the object is the origin’s facts, not the cache’s copies. **Not data-quality dashboards.** There is no dashboard: the semantics live in the query path, which is the difference between an opinion and a type. **Not automatic deletion.** Compaction demotes; it does not destroy — though it composes with retention law (Section 5.6), minimization is a policy that rides the same clock. **Not a conflict resolver.** Divergence events surface conflicts with decayed confidences; resolution is a downstream, often human, decision — the system’s job is to make the conflict visible before it costs a delivery or a dosage.

8 Objections, confronted

“The metadata overhead is prohibitive.” Four columns per fact and read-time derivation of confidence — a rounding error against the indexes every operational table already carries. The expensive-looking component, the verification scheduler, spends *budgeted* money replacing the unbudgeted money currently spent on the consequences (undeliverable mail, failed payments, wrong-door field visits, compliance findings). The objection prices the ledger and forgets to price the rot.

“Half-lives are unknowable per attribute.” They are *estimated*, from the organization’s own logs, with censoring-aware survival statistics (Section 5.2) — and the estimator’s honest output includes its uncertainty, which the display contract renders. The alternative is not perfect knowledge; it is the current regime of *zero* knowledge, whose error rate the B2B hygiene market quietly monetizes.

“Finance already solved staleness.” For quotes, with hard domain bounds, in engines built for one asset class — the existence proof that decay is engineerable, and simultaneously the proof that it has never been *generalized* (no CRM, no EHR, no ERP offers quote-machine staleness discipline for its own facts). Domain success without generalization is the landscape’s signature shape, and this program’s opportunity.

“Just update the data.” Verification costs money and attention; “just update” is the instruction that has no budget, no ranking, and no admission control — the verification economy of Section 2.4 is the missing allocation layer between the instruction and the spend. Everything schedulable gets scheduled; nothing schedulable gets done.

“Stale-but-stable dimensions will be flagged forever.” Facts whose true hazard is near zero (blood types, national identifiers) *should* verify cheaply and rarely — the hazard estimator assigns them long half-lives automatically, and their verification verbs are batch checks that cost pennies. The policy calibrates; the objection assumes it cannot.

9 Limitations

Hazard heterogeneity within a class is real — “address” in a student city and “address” in a retirement town share a column and not a biology; class-conditional estimators (segmented by entity covariates) mitigate at the cost of sample size, and the spec leaves the segmentation depth to the operator. Adversarial staleness — a party deliberately maintaining a false-but-fresh front — defeats time-based decay entirely; freshness is evidence, not proof, and the fabric composes with (but does not replace) source-trust machinery. Cold-start with thin update history yields wide intervals; the honest response is wide display tiers, not fake precision. The join-decay independence assumptions are approximations that entity-level maintenance half-lives only partially repair; the algebra’s goal is calibrated-enough propagation, not probabilistic rigor. And the display burden is a genuine UX cost — tiered rendering must not become tiered nagging — which is why the display contract inherits P-015’s calibrated minimalism rather than inventing a new anxiety grammar. Citations from domain knowledge carry the program’s *verification-queued* mark and pass the live-source ledger before final release.

10 Conclusion

The database is the only major information artifact that stores facts about the world without storing their perishability. Food has dates; medicine has lots; journalism has corrections; markets have staleness bounds wired into matching engines with regulatory teeth. Software — the industry with the most facts, the most joins, and the most downstream decisions per fact — has an eternity clause inherited from a 1970 model that never had to think about it, and a hygiene market that bills monthly for the consequences. The half-life semantics specified here are small, composable, and estimable from logs every operator already keeps: decay declarations, read-time confidence, an honest join algebra, a budgeted verification scheduler, staleness as an outage event, demotion as a storage state. The promise every stored value implies — *this is still true*

— becomes a typed, measurable, schedulable property, and the undeliverable package becomes the relic it should always have been. Facts expire. The systems that store them should finally know that.

References

Verification-queued marks follow the program’s citation-honesty convention.

1. Snodgrass, R. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, 1999.
2. International Organization for Standardization. *ISO/IEC 9075:2011 SQL, Part 2: temporal features (system-versioned tables)*.
3. Fielding, R. et al. *RFC 7234: HTTP/1.1 Caching*. IETF, 2014.
4. Redis Ltd. *Expiration and TTL semantics*. Documentation. (*verification queued*)
5. dbt Labs. *Source freshness configuration*. Documentation. (*verification queued*)
6. European Union. *GDPR Article 5(1)(d) — accuracy principle; Article 5(1)(e) — storage limitation*. 2016.
7. FINRA / exchange rulebooks on stale quotes and market data integrity. (*verification queued*)
8. B2B contact-data decay whitepapers, enrichment vendors. (*verification queued; vendor-sourced figures*)
9. Kalashnikov, L., Chen, Z., et al. on entity resolution and temporal aspects. (*verification queued*)
10. Cox, E., Seaman, C. on data quality dimensions in warehousing. (*verification queued*)
11. Aalen, O., Borgan, Ø., Gjessing, H. *Survival and Event History Analysis* (censoring-aware estimation). Springer.
12. Gittins, J., Glazebrook, K., Weber, R. *Multi-Armed Bandit Allocation Indices* (index policies for sequential allocation). Wiley.
13. Codd, E. F. *A Relational Model of Data for Large Shared Data Banks*. CACM 13(6), 1970.
14. p-rick research program. *P-001 The Personal Event Bus; P-006 The Attention Scheduler; P-015 The Uncertain Document; P-016 The Defaults Ledger*. 2026.
15. Woodall, P., et al. on data quality and maintenance-cost literature. (*verification queued*)
16. Dhamija, R., Dussault, L. “The seven flaws of identity management: usability and security.” *IEEE Security & Privacy*, 2008 (staleness of contact/recovery channels).